MittiMitra: Smartphone-Based Hybrid AI for Real-Time Soil Analysis and Advisory

Archana Prabhakaran Nair, Ceija, Riya Jaiswal, Parlapally Rithika, and Swaminathan J*
Dept. of Computer Science and Engineering
Amrita Vishwa Vidyapeetham, Amritapuri
Kollam, India
{archanaprabhakaran65, ceijajebasingh, jaiswaleshu61, rithika7337}@gmail.com
*Corresponding Author: swaminathanj@am.amrita.edu

***Abstract*—The Indian agricultural sector faces a critical "Diagnostic Latency Gap," where the time-intensive nature of centralized wet-chemistry labs compels smallholder farmers to practice indiscriminate fertilization, leading to systemic soil degradation. This paper presents *MittiMitra*, a symmetric hybrid-AI decision support system designed to democratize precision agriculture for resource-constrained edge environments.**

The proposed architecture introduces a novel multi-modal fusion engine that orchestrates on-device computer vision with cloud-based generative reasoning. At the edge, a quantized MobileNetV2 architecture (INT8) executes real-time lithological classification of nine distinct soil types with 91.4% accuracy and sub-150ms inference latency, ensuring functionality in zero-connectivity zones. Upon network availability, this visual telemetry is asynchronously fused with pedological data from the ISRIC SoilGrids API and hyper-local atmospheric variables from Open-Meteo. This context vector drives a Llama-3-70B inference engine to generate scientifically rigorous, dialect-specific agronomic advisories.

Beyond diagnostics, the system establishes a comprehensive "Digital Enclave" for the farmer, integrating a cryptographically secure document locker, a pivot-based agricultural unit converter, and a state-aware market intelligence tracker. By reducing the cognitive and logistical barriers to soil testing, MittiMitra facilitates a scalable paradigm shift from intuition-based farming to evidence-driven sustainable agriculture.

***Index Terms*—Edge Artificial Intelligence, Generative AI, Precision Agriculture, Soil Health Diagnostics, TensorFlow Lite, Decision Support Systems, Digital Inclusion, Sustainable Farming.**

I. Introduction

The sustainability of the global agricultural ecosystem is increasingly compromised by anthropogenic soil degradation. In the Indian subcontinent, the legacy of the Green Revolution has precipitated a crisis of "Nutrient Mining," where farmers, lacking precise soil intelligence, engage in "blind fertilization." This asymmetry of information has led to severe ecological consequences, including groundwater eutrophication, soil acidification, and a stagnation in crop productivity despite increased input costs.

The fundamental bottleneck preventing the adoption of Precision Agriculture (PA) in developing nations is the "Diagnostic Latency Gap." Traditional wet-chemistry laboratories operate on a centralized model that is inherently unscalable; a typical Soil Health Card (SHC) takes 14–30 days to generate. For a farmer operating within tight sowing windows, this latency renders the data historically interesting but agronomically useless. Furthermore, when results do arrive, they suffer from an "Interpretation Gap." Data presented in stoichiometric units (e.g., *"Organic Carbon: 0.3%"*) remains abstract and non-actionable for a demographic where functional literacy is variable.

Existing digital interventions have attempted to mitigate this but fall into two suboptimal categories: (1) *Cloud-Native Systems* that require high-bandwidth connectivity, failing in rural "shadow zones," or (2) *Static Offline Tools* that lack generative reasoning capabilities.

This paper proposes **MittiMitra** (Soil Friend), a symmetric Hybrid-AI Decision Support System (DSS) designed to bridge these digital and cognitive divides. Unlike predecessors, MittiMitra treats the smartphone not merely as a display terminal but as an edge-computing diagnostic instrument. By orchestrating a quantized Convolutional Neural Network (MobileNetV2) locally, the system provides zero-latency lithological classification of soil types (e.g., *Regur, Alluvial*) even in total isolation. Upon network restoration, this edge-intelligence is asynchronously fused with cloud-based geospatial chemistry (ISRIC) and atmospheric models (Open-Meteo) to drive a Large Language Model (Llama-3), generating advisory services that are scientifically rigorous yet dialectically accessible.

A. Key Contributions

The primary contributions of this work are as follows:

- **Hybrid-AI Orchestration:** We propose a novel "Symmetric Data Fusion" architecture that combines offline computer vision (Edge AI) with online generative reasoning (Cloud AI), ensuring system viability in intermittent network conditions.
- **Cognitive Accessibility:** The integration of a multilingual voice assistant and a neuro-inclusive interface (OpenDyslexic font support) to democratize access for low-literacy farmers.
- **Administrative Digitization:** The development of a cryptographically secure "Digital Document Locker" and a pivot-based "Unit Converter" to reduce the logistical friction of land management.

- **State-Aware Scheduling:** A battery-efficient background task manager that aligns data fetching with the circadian rhythms of agricultural decision-making.

II. Related Work and Gap Analysis

The domain of Digital Agriculture (DA) has witnessed a proliferation of mobile-based advisory systems. However, a critical analysis reveals that existing solutions largely operate in silos, addressing isolated aspects of farm management while neglecting the holistic socio-technical reality of the Indian smallholder.

A. Critique of Existing Literature

Early interventions, such as *SoilNutro* [8], attempted to digitize advisory services using standard agronomic look-up tables. While effective for general advice, they suffer from "Pedological Generalization," failing to account for the high spatial variability of soil nutrients in fragmented Indian landholdings.

The *Slakes* framework [9] pioneered "visual soil diagnostics" using smartphone cameras to analyze aggregate stability. However, its scope is strictly limited to physical structure, lacking the capability to infer chemical fertility (NPK) or integrate real-time atmospheric data.

More advanced platforms like *Farmonaut* [11] leverage satellite telemetry (Sentinel-2) for Soil Organic Carbon estimation. While effective for macro-level monitoring, these "Cloud-Native" systems require high-bandwidth connectivity, creating a "digital exclusion zone" for farmers in remote areas.

B. Gap Analysis

Table I highlights the deficiencies in current State-of-the-Art (SOTA) solutions which MittiMitra explicitly resolves.

TABLE I: Feature Matrix: MittiMitra vs. Existing SOTA

Feature	Slakes	Farmonaut	SoilNutro	MittiMitra
Edge-AI Inference	✓	×	×	✓
Offline Persistence	×	×	✓	✓
Hyper-local Weather	×	✓	×	✓
Admin. Tools (Locker)	×	×	×	✓
Neuro-Inclusive UI	×	×	×	✓

Our Contribution: MittiMitra bridges these gaps via a **Hybrid-AI Orchestration** layer. Unlike Farmonaut, our quantized MobileNetV2 model ensures diagnostic capability in zero-connectivity zones. Furthermore, by integrating a **Digital Document Locker** and **Neuro-Inclusive UI**, we transform the application from a mere diagnostic tool into a comprehensive operational platform.

III. System Architecture and Design Implementation

The MittiMitra architecture is engineered as a distributed, partition-tolerant system designed to function reliably in the intermittent network conditions characteristic of rural India. Unlike traditional thin-client agricultural applications that rely entirely on server-side processing, our proposed system implements a *Symmetric Multi-Modal Data Fusion* architecture. This design distributes the computational load between the edge device (smartphone) and the cloud infrastructure, adhering to the *Offline-First* principle.

A. Architectural Topology

The system is organized into a three-tier hierarchical topology, ensuring a clear separation of concerns between perception, logic, and storage.

1) Tier 1: Edge-Intelligence & Computer Vision Layer: The primary diagnostic interface is the on-device inference engine. To accommodate the hardware constraints of budget smartphones, we utilize a MobileNetV2 architecture as the backbone. This model is optimized via post-training quantization (INT8), reducing the model size to under 4MB without significant accuracy degradation.

The input pipeline utilizes a dedicated `ImageCompressor` module to manage memory allocation. Raw sensor data captured via the CameraX API is subjected to a preprocessing routine that standardizes the bitmap to a $224 \times 224 \times 3$ tensor using bilinear interpolation. This normalized tensor is fed into a local TensorFlow Lite interpreter, classifying soil into nine lithological categories with a mean inference latency of ≈ 150ms. This local classification loop ensures that the diagnostic capability is independent of cellular connectivity.

2) Tier 2: Reactive Persistence Layer (Offline-First): To manage data consistency across volatile network states, the system employs the **Repository Pattern**. We utilize the Room Persistence Library (an abstraction over SQLite) to instantiate a local database, acting as the Single Source of Truth (SSOT) for the device.

Simultaneously, the Firestore SDK is configured with offline persistence enabled. This allows the application to function in an "Eventual Consistency" mode. The UI employs an *Optimistic Concurrency Control* strategy—reflecting changes immediately in the view layer—while a background thread manages the synchronization handshake with the cloud. This ensures that a farmer can save a soil report in a dead zone, and the data will seamlessly synchronize once the device re-enters network coverage.

3) Tier 3: Asynchronous Cloud Fusion Engine: Upon network detection, the application triggers a parallelized data fetching routine using **Retrofit** clients. This utilizes a **Micro-Service Client** approach, where distinct interfaces handle disparate data sources:

- **Pedological Service:** Fetches vertical soil profile data (pH, CEC) from the ISRIC SoilGrids API based on geospatial coordinates.
- **Atmospheric Service:** Retrieves hyper-local weather variables (precipitation probability, evapotranspiration) from Open-Meteo.
- **Market Intelligence Service:** Queries real-time government commodity prices via the Mandi API.

These disparate data vectors are fused into a context-rich prompt, which is processed by the Llama-3 inference engine to generate a holistic agronomic advisory.

B. Software Design Patterns

To ensure maintainability, testability, and resource efficiency on constrained devices, the application strictly adheres to established software engineering patterns:

1) **Singleton Pattern:** Critical resource-heavy components, specifically the `RetrofitClient` network handler and the `MittiMitraDatabase` instance, are implemented as thread-safe Singletons using the 'synchronized' keyword. This prevents memory leaks and ensures a singular connection pool for network requests and database transactions.
2) **Factory Pattern:** The security layer utilizes a Provider Factory model to instantiate the `AppCheck` service. The system dynamically selects between `DebugAppCheckProviderFactory` (during development/CI) and `PlayIntegrityAppCheckProviderFactory` (in production signed builds). This abstraction allows for rigorous security in the wild without hampering the developer experience.
3) **Observer Pattern:** The UI layer employs reactive observers on Firestore collections and Room LiveData. For instance, the dashboard attaches a real-time listener to the User Profile document. This allows the UI to dynamically re-render components—such as the "Welcome" message or language preference—immediately upon data mutation, decoupling the View from the Model.
4) **Adapter Pattern:** To efficiently render dynamic lists such as the "Crop History" or "Market Prices," we implement custom `RecyclerView.Adapter` classes. These adapters act as a bridge between the underlying data lists and the UI components, implementing view recycling to minimize memory churn during scrolling.

C. Security and Integrity Layer

Given the potential for this system to integrate with government subsidy schemes, ensuring data provenance and transport security is critical.

1) Cryptographic Attestation: The application integrates **Firebase App Check** to prevent API abuse. The `MittiMitraApp` class initializes the `Play Integrity` provider, which performs a cryptographic attestation of the device's hardware-backed key store. This ensures that requests originate from a genuine, unmodified binary running on a trusted Android device, effectively mitigating "Sybil attacks" where botnets might flood the system with synthetic soil reports.

2) Transport Layer Security: Network traffic is secured via a strict `NetworkSecurityConfig`. The application enforces HTTPS for all external communications and explicitly disables clear-text traffic. This prevents Man-in-the-Middle (MitM) attacks, ensuring that sensitive pedological and user profile data remains encrypted in transit.

3) Scoped Storage Isolation: To protect user documents (e.g., land deeds), the system utilizes Android's **Scoped Storage** model. Sensitive images are stored in the application's private internal directory rather than public shared storage, ensuring they are inaccessible to other malicious applications installed on the device.

IV. Methodology and Algorithmic Implementation

The operational efficacy of MittiMitra is predicated on the rigorous implementation of several domain-specific algorithms. These modules are engineered to balance computational efficiency with the high variability of agricultural contexts.

A. A. Pivot-Based Unit Standardization Algorithm

A pervasive challenge in the Indian agrarian landscape is the lack of standardized land measurement units. Regional units such as *Bigha* vary not just between states, but between districts (e.g., a Bigha in Bihar is $2,529m^2$ while in Assam it is $1,338m^2$). To resolve this fragmentation without incurring the $O(N^2)$ complexity of a direct conversion matrix, the `UnitConverterActivity` implements a **Pivot-Based Normalization Architecture**.

1) Mathematical Formalization: Let $U = \{u_1, u_2, \ldots, u_n\}$ be the set of all supported regional units. We define a singular pivot unit, u_{pivot} = Hectare. For every unit $u_i \in U$, we define a normalization coefficient λ_i such that:

$$Value(u_{pivot}) = Value(u_i) \times \lambda_i \tag{1}$$

The conversion from a source unit u_{src} to a target unit u_{tgt} is performed via a two-step transitive operation $f : \mathbb{R} \to \mathbb{R}$:

$$f(x) = x \times \lambda_{src} \times \frac{1}{\lambda_{tgt}} \tag{2}$$

This approach reduces the space complexity from $O(N^2)$ to $O(N)$, ensuring that adding a new regional unit requires defining only one new scalar coefficient rather than N new relationships. The implementation utilizes `BigDecimal` types to prevent floating-point underflow errors during high-precision land subdivisions.

B. B. State-Aware Background Scheduling

Agricultural intelligence is time-sensitive. To automate the delivery of advisories, the application implements a "State-Aware" scheduling policy utilizing the **Android Jetpack WorkManager API**. This system is designed to align with the circadian and operational rhythms of farming while strictly adhering to the device's battery optimization protocols (Doze Mode).

1) The Circadian Weather Cycle: Irrigation planning is typically performed at dawn. Consequently, the application schedules a `PeriodicWorkRequest` tagged as `SmartWeatherWork` with a recurrence interval $\Delta t = 12$ hours. To ensure reliability, we apply a `Constraints.Builder` to enforce execution only when:

- `setRequiredNetworkType(CONNECTED)`: Ensures fresh data fetch.
- `setRequiresBatteryNotLow(true)`: Prevents execution during critical battery drain.

The worker executes a decision tree logic defined in `NotificationWorker.java`:

$$Action = \begin{cases} \text{Notify: "Postpone Irrigation"} & \text{if } P_{rain} > \tau_{rain} \\ \text{Notify: "Mulching Advised"} & \text{if } T_{max} > \tau_{temp} \wedge H_{rel} < 30\% \\ \text{Silent Cache Update} & \text{otherwise} \end{cases} \tag{3}$$

Where $P(rain)$ is precipitation probability, T_{max} is ambient temperature, and H_{rel} is relative humidity. The thresholds are empirically defined in `AppConstants` as $\tau_{rain} = 60\%$ and $\tau_{temp} = 35°C$.

2) High-Frequency Alert Loop: A secondary, high-frequency worker operates on a 15-minute interval to poll for "Force-Push" government notifications (e.g., cyclone warnings). This separation of concerns—splitting the "Agronomic Loop" (12h) from the "Safety Loop" (15m)—ensures that the application minimizes radio wake-ups, thereby extending the device's operational lifespan in field conditions.

C. C. Secure Digital Locker Implementation

The `DocumentsActivity` functions as an encrypted digital vault for sensitive records (e.g., *Khasra-Khatauni* land deeds). This module addresses two conflicting requirements: high visual fidelity for text legibility and low storage footprint for budget devices.

1) Adaptive Compression Algorithm: We implemented a custom `ImageCompressor` class that employs a dynamic scaling algorithm. Given an input image I with dimensions $W \times H$, the algorithm calculates a scaling factor S:

$$S = \min\left(\frac{1024}{W}, \frac{1024}{H}\right) \tag{4}$$

If $S < 1$, the image is downsampled using bilinear interpolation to a maximum bounding box of $1024px$. The resulting bitmap is compressed using the JPEG codec at $Q = 90$ quality.

$$Size_{final} \approx \frac{Size_{raw}}{Compression(Q)} \times S^2 \tag{5}$$

This ensures that a typical 5MB camera capture is reduced to $\approx$ 300KB while retaining sufficient resolution for OCR (Optical Character Recognition) verification.

2) Scoped Storage Architecture: To ensure security, documents are not stored in the public gallery. Instead, they are written to the application's **Internal Scoped Storage** directory, making them inaccessible to other malicious applications. The metadata is indexed in a Room Database (`DocumentDao`) using a composite primary key $\{User_{ID}, Doc_{Type}, Timestamp\}$, allowing for $O(\log N)$ retrieval complexity.

D. D. Neuro-Inclusive User Interface Design

Recognizing that 28.4% of the rural population may suffer from undiagnosed reading difficulties or visual impairments, MittiMitra implements a **Dynamic Resource Injection** system.

1) Runtime Typeface Injection: The `AccessibilityActivity` allows users to toggle the "OpenDyslexic" typeface globally. This is achieved by overriding the `attachBaseContext()` method in the `BaseActivity` class.

1) The system intercepts the standard Android `Configuration`.
2) It injects a custom `ContextWrapper` that overrides the default font family parameters.
3) It forces a comprehensive UI recreation (`recreate()`), propagating the dyslexic-friendly font to all child views (TextViews, Buttons, Toolbars) without requiring individual XML modifications.

2) WCAG-Compliant Theming: The `ThemeSettingsActivity` provides a high-contrast "Tricolor High-Vis" mode. We utilized the Web Content Accessibility Guidelines (WCAG) 2.1 AA standard to define our color palette:

- **Primary (Saffron):** Hex `#FF9933` (Contrast Ratio 3.1:1 against white).
- **Secondary (Green):** Hex `#138808` (Contrast Ratio 5.2:1 against white).
- **Typography:** All critical reading text is rendered in `#121212` (Obsidian Black) to ensure maximum legibility under direct sunlight.

V. Detailed Module Implementation

The MittiMitra ecosystem is architected as a comprehensive "Digital Enclave," extending beyond simple soil diagnostics to cover the entire agricultural value chain. The system is compartmentalized into three functional domains: Agronomic Intelligence, Socio-Economic Utilities, and Digital Identity Management.

A. Domain A: Core Agronomic Intelligence

1) Pre-emptive Weather Alert System: Agriculture is inherently climate-dependent. To mitigate risks associated with sudden climatic shifts, the system implements a "Pre-emptive Warning Protocol." A background worker utilizes the Android WorkManager API to poll hyper-local atmospheric data every 15 minutes.

Operational Logic: The system monitors precipitation probability (P_{rain}) and wind velocity (V_{wind}). If $P_{rain} > 65\%$ or $V_{wind} > 25$ km/h, a high-priority system notification is triggered, advising the user to secure standing crops or delay fertilizer application to prevent leaching.

2) The "Plant Doctor" (Pathology Scanner): Early disease detection is critical for reducing pesticide overuse. This module employs a specialized Computer Vision pipeline distinct from the soil classifier.

Visual Analysis:High-resolution images of foliar symptoms (e.g., leaf blight, rust) are captured via the camera hardware. Inference Engine: These images are compressed using an adaptive scaling algorithm and processed against a disease detection model. This allows farmers to identify pathogens in real-time and receive immediate chemical or organic remediation strategies.

3) Lithological Soil Classification: The core engine of the application transforms the smartphone into a handheld pedometer.

Edge-AI Execution: Utilizing a quantized TensorFlow Lite model (MobileNetV2), the system performs texture analysis on 224×224 pixel input tensors. Probabilistic Output:The model computes a softmax probability distribution over nine soil classes (e.g., Black, Red, Laterite). The class with the highest confidence score is persisted locally, triggering the retrieval of relevant crop suitability data.

4) Precision Irrigation Advisory: To address water scarcity, the application calculates an "Evapotranspiration Deficit."

Algorithmic Logic: By fusing real-time temperature and humidity data, the system estimates the rate of soil moisture loss. If the ambient temperature exceeds $30°C$ with humidity below 40%, an advisory is generated recommending evening irrigation to minimize evaporative losses.

5) Phenological Crop Calendar: This module tracks the biological lifecycle of cultivated crops.

Timeline Tracking: Upon registering a sowing date, the system calculates "Days After Sowing" (DAS). Stage-Specific Advice: It maps the DAS to specific phenological stages (Vegetative, Flowering, Grain Filling), providing timely reminders for nutrient application (e.g., "Day 45: Top dressing of Urea") customized to the crop's current growth phase.

6) Generative Advisory Interface ("Get Advice"): Moving beyond static FAQs, this module leverages Large Language Models (LLM) to answer context-specific queries.

Prompt Engineering: The system constructs a hidden context vector containing the user's soil type, location, and current weather. This vector is fed into the Llama-3-70B model to generate agronomic advice that is scientifically grounded yet linguistically accessible.

7) "Kisaan Sahaayak" (AI Chatbot): To support low-literacy users, a conversational interface is implemented. Contextual Memory: The chatbot maintains a sliding window of the conversation history, allowing it to remember previous interactions (e.g., "What fertilizer did you suggest for that crop?"). This history is persisted in a local database to ensure continuity across sessions.

8) Longitudinal History Tracking: To monitor soil health trends over time, the system maintains an immutable ledger of all performed scans.

Data Visualization: Previous analysis reports are displayed in reverse chronological order, allowing farmers to visualize changes in soil texture or organic carbon estimates across different planting seasons.

B. Domain B: Socio-Economic Utilities

1) Real-Time Mandi (Market) Intelligence: To address the "Information Asymmetry" that often forces farmers into distress selling, this module provides market transparency.

Data Aggregation: The application connects to government e-NAM and Agmarknet endpoints to fetch live commodity prices. Visual Analytics: Users can view the Minimum, Maximum, and Modal prices for specific crops in their local district, empowering them to negotiate better rates with intermediaries.

2) Secure Document Locker: Administrative security is handled via an encrypted digital vault.

Scoped Storage Security: Sensitive documents (e.g., land deeds, insurance papers) are stored within the application's private internal storage directory, rendering them inaccessible to other apps. Metadata Indexing: Files are indexed by category (Identity, Land Record), allowing for $O(1)$ retrieval during official inspections or bank loan applications.

3) Pivot-Based Land Unit Converter: Given the fragmentation of Indian land measurement units, a normalization tool is critical. Standardization Algorithm: The system uses "Hectare" as a mathematical pivot. Regional units (Bigha, Guntha, Kanal) are first converted to Hectares, then to the target unit, ensuring floating-point precision is maintained across diverse regional standards.

4) Support & Offline Knowledge Base: To ensure utility in disconnected environments, a static "Offline Tips" module provides best practices for sustainable farming, stored locally as JSON assets. Additionally, a direct "Call Kisaan" feature enables one-touch escalation to the National Kisan Call Center via telephony intent integration.

5) Government Scheme Repository: To bridge the "Awareness Gap" regarding state and central subsidies, this module implements an offline-first repository. Architectural Logic:The system parses a local JSON asset (`schemes_data.json`) containing detailed metadata on schemes like *PM-Kisan* and *Pradhan Mantri Fasal Bima Yojana*. By bundling this data within the APK, we ensure that critical financial information is accessible even in off-grid locations.Eligibility Filtering: The `SchemeRepository` class implements a filtering logic that correlates scheme criteria against the user's persisted profile (e.g., Landholding < 2 Hectares). This "Eligibility Engine" ensures that farmers are presented only with relevant financial instruments, reducing the cognitive load of navigating complex bureaucratic documentation.

C. Domain C: Accessibility and Inclusion

1) Linguistic Parity Engine: The application supports over 16 Indic languages. A custom Context Wrapper intercepts

the application's base context during initialization, injecting locale-specific resource files (e.g., Hindi, Telugu, Tamil, Manipuri, Malayalam, Kannada etc.) at runtime without requiring an application restart.

2) Neuro-Inclusive Accessibility: Recognizing the prevalence of undiagnosed reading difficulties, an accessibility toggle allows users to inject the *OpenDyslexic* typeface globally. Furthermore, high-contrast "Tricolor" themes (Saffron/Green) are implemented to adhere to WCAG 2.1 AA standards for outdoor visibility.

3) Data Sovereignty: The "Manage Data" module empowers users with full control over their digital footprint. It includes functionality to clear local caches or export the entire soil analysis database as a CSV file, facilitating data portability for external analysis or government reporting.

VI. Security and Integrity Verification

Given the potential for this system to integrate with government subsidy schemes, ensuring data provenance is critical. The application integrates the Firebase App Check SDK to prevent API abuse.

In the production environment, the `MittiMitraApp` class initializes the `PlayIntegrityAppCheckProviderFactory`. This service performs a cryptographic attestation of the device's hardware-backed key store and the application's binary signature. This ensures that the soil data uploaded to the server originates from a genuine, unmodified version of the MittiMitra application running on a trusted Android device, thereby preventing "sybil attacks" where bots might flood the system with fake soil reports.

VII. Results and Performance Analysis

A. Model Accuracy

The MobileNetV2 architecture was evaluated on a held-out test set of localized soil images. As shown in Table II, the model demonstrates high fidelity in distinguishing critical soil types.

TABLE II: Classification Metrics by Soil Type

Soil Type	Precision	Recall	F1-Score
Alluvial	0.92	0.91	0.915
Black (Regur)	0.94	0.93	0.935
Red	0.89	0.88	0.885
Laterite	0.88	0.87	0.875
Overall	**0.91**	**0.90**	**0.905**

B. System Latency

The "Cold Start" time for the generative advisory (from scan to voice output) averaged 4.2 seconds on a 4G network. Crucially, the offline lithological classification averaged **148ms** on a standard Android device, validating our claim of zero-latency edge diagnostics.

VIII. Conclusion

MittiMitra demonstrates that the "Digital Divide" in agriculture is not just about connectivity, but about cognitive accessibility. By fusing Edge AI (MobileNetV2) for immediate diagnostics with Cloud AI (Llama-3) for agronomic reasoning, we have created a decision support system that is both scientifically rigorous and deeply empathetic to the farmer's reality. The modular architecture establishes a robust blueprint for the next generation of Agri-Tech applications.

References

[1] T. Hengl et al., "SoilGrids250m: Global gridded soil information based on machine learning," *PLOS ONE*, vol. 12, no. 2, 2017.

[2] Open-Meteo, "Historical Weather API," 2024. [Online]. Available: https://open-meteo.com

[3] M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *IEEE CVPR*, 2018, pp. 4510-4520.

[4] TensorFlow, "TensorFlow Lite Guide," Google Developers, 2024. [Online].

[5] Meta AI, "Llama 3 Model Card," 2024. [Online]. Available: https://llama.meta.com/

[6] Govt of India, "Soil Health Card Scheme," Ministry of Agriculture & Farmers Welfare, 2015.

[7] Android Developers, "WorkManager — Android Jetpack," Google, 2024.

[8] J. Smith and R. Kumar, "SoilNutro: Mobile App for fertilizer recommendation," *Int. Journal of Agri. Informatics*, vol. 5, no. 2, 2019.

[9] K. S. Flynn et al., "Slakes: A smartphone application to measure soil aggregate stability," *Soil and Tillage Research*, vol. 196, 2020.

[10] A. Patel et al., "GeaGrow: AI Fertilizer Optimizer for Africa," *Proc. of IEEE Global Humanitarian Technology Conference*, 2021.

[11] Farmonaut, "Satellite Based Crop Health Monitoring," 2023. [Online]. Available: https://farmonaut.com

[12] Android Developers, "Save data in a local database using Room," Google, 2024.

[13] Square Inc., "Retrofit: A type-safe HTTP client for Android and Java," 2024.

[14] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[15] Google Play, "Play Integrity API," Android Developers, 2024.